

GoMuseum开发指南 - 前置章节

项目概述

GoMuseum是一个现代化的博物馆数字化平台，致力于为用户提供沉浸式文化体验。本项目采用现代Web技术栈，支持多媒体展示、互动体验和离线访问功能。

项目目标

- 构建高性能、可扩展的博物馆数字化平台
- 提供优质的用户体验和无障碍访问
- 支持多设备、多场景的使用需求
- 实现内容管理和数据分析功能

开发规范

代码规范

1. 命名规范

- 文件命名: 使用kebab-case, 如 `user-profile.tsx`
- 变量命名: 使用camelCase, 如 `userName`
- 常量命名: 使用SCREAMING_SNAKE_CASE, 如 `API_BASE_URL`
- 组件命名: 使用PascalCase, 如 `UserProfile`
- 函数命名: 使用camelCase, 动词开头, 如 `getUserData()`

2. 目录结构规范

```
src/
├── components/    # 可复用组件
│   ├── common/   # 通用组件
│   ├── forms/    # 表单组件
│   └── ui/        # UI基础组件
├── pages/         # 页面组件
├── hooks/         # 自定义Hooks
├── utils/         # 工具函数
├── services/      # API服务
├── types/         # TypeScript类型定义
├── constants/     # 常量定义
├── assets/        # 静态资源
└── tests/         # 测试文件
```

3. 代码质量工具配置

Black格式化配置 (pyproject.toml)

```
toml
```

```
[tool.black]
line-length = 88
target-version = ['py311']
include = '\.pyi?$'
extend-exclude = '''
/(
# directories
\.eggs
| \.git
| \.hg
| \.mypy_cache
| \.tox
| \.venv
| build
| dist
)/
'''
```

ESLint配置 (.eslintrc.js)

```
javascript

module.exports = {
  extends: [
    '@typescript-eslint/recommended',
    'prettier/@typescript-eslint',
    'plugin:react/recommended',
    'plugin:react-hooks/recommended'
  ],
  rules: {
    '@typescript-eslint/explicit-function-return-type': 'error',
    '@typescript-eslint/no-unused-vars': 'error',
    'react/prop-types': 'off',
    'react/react-in-jsx-scope': 'off'
  }
}
```

Prettier配置 (.prettierrc)

```
json

{
  "semi": true,
  "trailingComma": "es5",
  "singleQuote": true,
  "printWidth": 80,
  "tabWidth": 2,
  "useTabs": false
}
```

测试规范

1. 测试文件命名

- 单元测试: *.test.ts 或 *.test.tsx
- 集成测试: *.integration.test.ts
- E2E测试: *.e2e.test.ts

2. 测试覆盖率要求

- 单元测试覆盖率: ≥80%
- 集成测试覆盖率: ≥70%
- 关键业务逻辑: 100%覆盖率

3. 测试结构规范

```
typescript
```

```
describe('ComponentName', () => {
  beforeEach(() => {
    // 测试前置条件
  });

  describe('when condition', () => {
    it('should behavior', () => {
      // Arrange
      // Act
      // Assert
    });
  });
});
```

DevOps CI/CD

Git分支管理策略

1. 分支结构

main (主分支)

- ├── staging (预发布分支)
- └── feature/ (功能分支)
 - ├── init/ (初始化功能)
 - │ ├── feature/init/project-setup
 - │ └── feature/init/basic-structure
 - └── bug-fix/ (Bug修复)
 - ├── feature/bug-fix/login-issue
 - └── feature/bug-fix/performance-fix

2. 分支命名规范

- 功能分支: `feature/init/{description}` 或 `feature/bug-fix/{description}`
- 发布分支: `release/v{version}`
- 热修复分支: `hotfix/{description}`

3. 分支保护规则

Main分支保护（独立开发者模式）

- 禁止直接推送
- 需要Pull Request (自我审查)
- 必须通过所有CI检查
- 必须更新到最新版本
- 删除分支保护时需要管理员权限

Staging分支保护（独立开发者模式）

- 允许直接推送（快速迭代）
- 必须通过基础CI检查
- 自动部署到staging环境

4. 提交信息规范

采用Conventional Commits规范：

<type>[optional scope]: <description>

[optional body]

[optional footer(s)]

类型定义：

- `feat`: 新功能
- `fix`: Bug修复
- `docs`: 文档更新
- `style`: 代码格式调整
- `refactor`: 代码重构

- `test`: 测试相关
- `chore`: 构建工具或辅助工具变动

示例：

```
feat(auth): add user authentication system
```

```
Implement JWT-based authentication with login/logout functionality
```

- Add login form component
- Integrate with backend API
- Add token storage and validation

```
Closes #123
```

CI/CD流程

1. CI触发规则

- **Pull Request**: 触发完整CI检查
- **Push到feature分支**: 触发基础检查
- **Push到staging**: 触发完整检查 + 部署到staging环境
- **Push到main**: 触发完整检查 + 部署到production环境

2. CI检查项目

```
yaml
```

```
# .github/workflows/ci.yml
name: CI
on:
  pull_request:
    branches: [main, staging]
  push:
    branches: [main, staging, 'feature/**']

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
      - run: npm ci
      - run: npm run lint
      - run: npm run test:coverage
      - run: npm run build

  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm audit
      - run: npm run security:check
```

3. CD部署流程

yaml


```
# .github/workflows/deploy.yml
name: Deploy
on:
  push:
    branches: [main, staging]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to staging
        if: github.ref == 'refs/heads/staging'
        run: ./scripts/deploy-staging.sh
      - name: Deploy to production
        if: github.ref == 'refs/heads/main'
        run: ./scripts/deploy-production.sh
```

环境管理

三环境架构

1. Local环境 (开发环境)

- 用途: 本地开发和调试
- 数据库: SQLite / 本地PostgreSQL
- 缓存: 本地Redis (可选)
- 文件存储: 本地文件系统
- 配置: `.env.local`

2. Staging环境 (预发布环境)

- 用途: 集成测试和用户验收测试
- 数据库: PostgreSQL (测试数据)

- 缓存: Redis
- 文件存储: AWS S3 (测试bucket)
- 配置: `.env.staging`
- 域名: `staging.gomuseum.com`

3. Production环境 (生产环境)

- 用途: 正式对外服务
- 数据库: PostgreSQL (生产数据)
- 缓存: Redis Cluster
- 文件存储: AWS S3 (生产bucket)
- CDN: CloudFront
- 配置: `.env.production`
- 域名: `gomuseum.com`

2025年技术栈版本清单

前端技术栈

```
json
{
  "node": "20.11.0",
  "npm": "10.4.0",
  "typescript": "5.3.3",
  "react": "18.2.0",
  "next.js": "14.1.0",
  "tailwindcss": "3.4.1",
  "framer-motion": "11.0.3",
  "three.js": "0.160.1",
  "@testing-library/react": "14.2.1",
  "jest": "29.7.0",
  "playwright": "1.41.2"
}
```

后端技术栈

```
json

{
  "python": "3.11.7",
  "django": "5.0.1",
  "django-rest-framework": "3.14.0",
  "postgresql": "16.1",
  "redis": "7.2.4",
  "celery": "5.3.4",
  "gunicorn": "21.2.0",
  "nginx": "1.24.0"
}
```

开发工具版本

```
json

{
  "docker": "25.0.2",
  "docker-compose": "2.24.5",
  "git": "2.43.0",
  "vscode": "1.86.0",
  "chrome": "121.0.0",
  "postman": "10.22.0"
}
```

环境依赖安装指导

1. 系统要求检查清单

```
bash
```

```
#!/bin/bash
# system-check.sh

echo "==== GoMuseum环境检查 ===="

# 检查操作系统
echo "操作系统: $(uname -s)"
echo "架构: $(uname -m)"

# 检查必需软件
check_command() {
    if command -v "$1" &> /dev/null; then
        echo "✅ $1: $(command -v "$1")"
        "$1" --version 2>/dev/null | head -n1
    else
        echo "❌ $1: 未安装"
    fi
}

check_command node
check_command npm
check_command python3
check_command docker
check_command git

echo "==== 检查完成 ===="
```

2. Node.js环境安装

```
bash
```

```
# 使用nvm安装Node.js
```

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
```

```
source ~/.bashrc
```

```
nvm install 20.11.0
```

```
nvm use 20.11.0
```

```
nvm alias default 20.11.0
```

```
# 验证安装
```

```
node --version
```

```
npm --version
```

3. Python环境安装

```
bash
```

```
# 使用pyenv安装Python
```

```
curl https://pyenv.run | bash
```

```
echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.bashrc
```

```
echo 'command -v pyenv >/dev/null || export PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.bashrc
```

```
echo 'eval "$(pyenv init -)"' >> ~/.bashrc
```

```
source ~/.bashrc
```

```
pyenv install 3.11.7
```

```
pyenv global 3.11.7
```

```
# 创建虚拟环境
```

```
python -m venv venv
```

```
source venv/bin/activate # Linux/Mac
```

```
# 或 venv\Scripts\activate # Windows
```

4. Docker环境安装

```
bash
```

Ubuntu/Debian

`sudo apt-get update`

`sudo apt-get install docker.io docker-compose-plugin`

macOS

`brew install --cask docker`

启动Docker服务

`sudo systemctl start docker`

`sudo systemctl enable docker`

验证安装

`docker --version`

`docker compose version`

5. 数据库环境安装

bash

```
# PostgreSQL
## Ubuntu/Debian
sudo apt-get install postgresql postgresql-contrib

## macOS
brew install postgresql
brew services start postgresql

# Redis
## Ubuntu/Debian
sudo apt-get install redis-server

## macOS
brew install redis
brew services start redis

# 验证数据库连接
psql --version
redis-cli ping
```

环境配置检查清单

Pre-Development Checklist

- ☐ Node.js 20.11.0已安装并配置为默认版本
- ☐ Python 3.11.7已安装在虚拟环境中
- ☐ Docker和Docker Compose已安装并运行正常
- ☐ PostgreSQL已安装并可连接
- ☐ Redis已安装并运行正常
- ☐ Git已配置用户信息和SSH密钥
- ☐ IDE/编辑器已安装推荐插件
- ☐ 环境变量文件已创建(.env.local)
- ☐ 项目依赖已安装(npm install & pip install)
- ☐ 数据库已创建并迁移完成
- ☐ 开发服务器可正常启动

- ☐ 测试套件可正常运行
- ☐ 代码质量工具配置正确
- ☐ 浏览器开发工具已准备就绪

监控告警

SonarCloud集成

1. 项目配置

properties

sonar-project.properties

sonar.projectKey=gomuseum

sonar.organization=your-org

sonar.sources=src

sonar.tests=src

sonar.test.inclusions=**/*.test.**,**/*.spec.*

sonar.coverage.exclusions=**/*.test.**,**/*.spec.**,**/node_modules/**

sonar.typescript.lcov.reportPaths=coverage/lcov.info

2. CI集成

yaml

GitHub Actions 中集成SonarCloud

- name: SonarCloud Scan

uses: SonarSource/sonarcloud-github-action@master

env:

GITHUB_TOKEN: \${ secrets.GITHUB_TOKEN }

SONAR_TOKEN: \${ secrets.SONAR_TOKEN }

3. 质量门禁标准

- 代码覆盖率: $\geq 80\%$
- 重复率: $\leq 3\%$

- 可维护性评级: A
- 可靠性评级: A
- 安全性评级: A
- 技术债务: ≤5天

CodeCov集成

1. 覆盖率报告生成

```
json

{
  "scripts": {
    "test:coverage": "jest --coverage --coverageReporters=lcov",
    "test:upload": "codecov -t $CODECOV_TOKEN"
  }
}
```

2. codecov.yml配置

```
yaml

coverage:
  status:
    project:
      default:
        target: 80%
        threshold: 2%
    patch:
      default:
        target: 70%
  ignore:
    - "**/*.test.*"
    - "**/*.spec.*"
    - "**/mocks/**"
```

Pre-commit Hooks

1. 安装配置

bash

```
npm install --save-dev husky lint-staged  
npx husky install
```

2. .husky/pre-commit

bash

```
#!/usr/bin/env sh  
.  
"$(dirname -- "$0")/_/husky.sh"  
  
npx lint-staged  
npm run test:staged
```

3. package.json配置

json

```
{  
  "lint-staged": {  
    "*.js,jsx,ts,tsx": [  
      "eslint --fix",  
      "prettier --write",  
      "git add"  
    ],  
    "*.md,json,yaml,yml": [  
      "prettier --write",  
      "git add"  
    ]  
  }  
}
```

4. 提交前检查项

- ☐ 代码格式化(Prettier)
- ☐ 代码质量检查(ESLint)
- ☐ 类型检查(TypeScript)
- ☐ 单元测试运行
- ☐ 提交信息格式验证
- ☐ 敏感信息扫描
- ☐ 依赖安全检查

告警通知配置

1. 告警级别定义

- **P0-紧急:** 生产环境服务中断, 需立即处理
- **P1-高优先级:** 核心功能异常, 24小时内修复
- **P2-中优先级:** 一般功能问题, 72小时内修复
- **P3-低优先级:** 优化建议, 下个版本处理

2. 通知渠道

- 邮件通知: 所有等级告警
- Slack通知: P0-P1告警
- 短信通知: P0告警
- 钉钉通知: 工作时间P1告警

3. 监控指标

- 构建成功率: >95%
- 测试覆盖率: >80%
- 代码质量评分: >B级
- 部署成功率: >98%
- 响应时间: <2秒

- 错误率: <1%

总结

本前置章节建立了GoMuseum项目的开发基础，包括：

1. **规范体系**: 统一的代码规范、测试规范和提交规范
2. **工具链**: 完整的代码质量工具和自动化检查
3. **环境管理**: 三环境架构和详细的安装指导
4. **CI/CD流程**: 自动化的构建、测试和部署
5. **监控体系**: 全方位的代码质量和性能监控

这些基础设施将为后续的项目开发提供强有力的支撑，确保项目的高质量交付。

下一步: 请确认本文档内容后，我将继续生成《GoMuseum开发指南-项目初始化.md》文档。